Syna

Offline P2P Messenger & Friend Finder

Feature Dimension	**Traditional Mesh Messengers**	**Our P2P Radar & Chat App**
Primary Focus	Text broadcasting & crisis messaging	Directional location tracking + off-grid chat
UI Experience	Standard 2D chat list view	**Dynamic 360° Compass Radar View**
Sensor Integration	Minimal (Network radio only)	**GPS Hardware + Magnetometer (Compass)**
Data CRUD Scope	Store & forward messages	Local waypoint logs, live coordinate patches, & chat storage

Proposal W9 -- 11/03

- 5 min present

Demo W14&15 -- 12/08, 12/15

- 10 min present

1. General Overview

Hook & Use Case

"What if you're hiking with friends, lose internet connectivity, and need to communicate or locate each other?"

During hiking, camping, festivals (crowded events), or other outdoor activities, users may experience limited or unavailable internet connectivity.

Pain Points

1. **Internet Dependency:** Conventional messaging applications rely on internet connectivity, making communication difficult in areas with poor or no network coverage.
2. **Limited Location Sharing:** Most location-sharing services require internet access to transmit locations and display maps.
3. **Difficulty Locating Friends:** GPS coordinates alone are not intuitive for finding someone's direction, especially in unfamiliar outdoor environments.

Proposed Solution

Our application enables nearby users to **communicate securely, share GPS locations, and find each other's direction without internet access.**

The application combines three core capabilities:

1. **Offline Messaging:** Exchange encrypted messages directly between nearby devices.
2. **Offline Location Sharing:** Share GPS coordinates and visualize friends' last known locations on an offline map.
3. **Friend Finder Compass:** Find the direction toward a friend's last shared location using a compass with an ambient edge-glow indicator that responds to the phone's orientation.

The application uses direct peer-to-peer connectivity, so communication requires devices to be within supported connection range. Previously received locations remain accessible when disconnected.

2. Functional Requirements (FR)

ID	Functional Requirement
FR-01	Users can register, log in, and manage their local profiles.
FR-02	Users can generate and scan QR codes to exchange public keys and add friends.
FR-03	Users can view, rename, and delete friends.
FR-04	Users can discover and connect to nearby devices without internet access.
FR-05	Users can send and receive encrypted one-to-one text messages offline.
FR-06	Users can view and delete locally stored chat history.
FR-07	Users can share their current GPS coordinates with connected friends.
FR-08	Users can view their location and all nearby connected friends' last shared locations as markers on an offline map.
FR-09	Users can view the distance and direction toward a selected connected friend's last shared location using a compass with an animated directional edge-glow indicator.
FR-10	Users can view connection status, message delivery status, and location timestamps.

3. Non-Functional Requirements (NFR)

gadinilai

ID	Non-Functional Requirement
NFR-01	Core messaging, location sharing, and compass features must work without internet connectivity.
NFR-02	Messages and location data must be encrypted during transmission using authenticated sessions.
NFR-03	Private keys must remain securely stored on the device and must never be transmitted.
NFR-04	Messages should be delivered within 3 seconds under stable nearby connectivity.
NFR-05	The application must provide an intuitive, responsive, and consistent UI.
NFR-06	Chat history, profiles, and contacts must persist after application restarts.
NFR-07	The application must handle disconnections, unavailable GPS, and denied permissions gracefully.
NFR-08	Location sharing must require explicit user consent.
NFR-09	The Friend Finder must display location freshness and indicate unreliable compass readings.

4. Tech Stack

Component	Technology
Language	Kotlin
UI Framework	Jetpack Compose + Material 3
Architecture	MVVM
Local Database	Room (SQLite)
P2P Communication	Google Nearby Connections API
Cryptography	Android Keystore + standard cryptographic libraries
QR Code	ZXing / ML Kit
Location	Android Location Services
Offline Maps	MapLibre + offline map tiles
Compass	Android Rotation Vector Sensor
Ambient Edge Glow	Jetpack Compose Canvas + Animations
Version Control	Git + GitHub

5. Job Distribution (3 Members)

Member	Responsibility	Features
Member 1: EL Networking & Security Layer	P2P connectivity, encryption, and low-level services	Nearby Connections API, device discovery, connection management, key generation/exchange, encrypted payload transmission, GPS and orientation sensor services
Member 2: AT Database & Business Logic	Data persistence, domain logic, and application architecture	Room (SQLite), local authentication, profile/contact/message CRUD, QR identity management, messaging logic, location sharing logic, distance/bearing calculations
Member 3: AMAF Frontend & UI/UX	Complete user interface and interaction design	Jetpack Compose screens, navigation, QR scanner UI, chat interface, offline MapLibre integration, map markers, compass visualization, animated directional edge glow

Shared Responsibilities:

- Integration testing and debugging
- UI/UX consistency and accessibility
- GitHub collaboration and code reviews
- Documentation and final presentation

6. Project Scope

MVP: Local identity, QR friend exchange, one-to-one encrypted offline messaging, chat history, contacts CRUD, offline location sharing, and directional Friend Finder.

Out of MVP: Multi-hop mesh networking (permission setuju/not relay hop), file sharing, group messaging, voice/video calls, cloud synchronization, turn-by-turn offline navigation

7. Implementation Plan (4 Weeks)

Strategy: Prioritize technical feasibility first, then parallel development, integration, and demo stabilization.

Nearby Connectivity → QR Identity & Authentication → Encrypted Messaging → Location Sharing → Friend Finder → Final Integration

Weekly Roadmap

Week	Focus	Key Deliverables
W1	Foundation & Technical Validation	Android project setup, architecture, Room schema, UI wireframes, successful offline P2P text exchange
W2	Core Messaging & Identity	QR friend exchange, authenticated encrypted messaging, chat UI, local authentication, chat history
W3	Location & Friend Finder	GPS sharing, offline MapLibre, friend markers, distance/bearing, compass + edge glow
W4	Integration & Finalization	End-to-end testing, bug fixes, UI polish, performance validation, presentation and live demo

Detailed Tasks & Dependencies

M1 = Networking & Security, M2 = Database & Business Logic, M3 = Frontend & UI/UX.

Week	Owner	Tasks	Dependencies
W1	M1	Nearby discovery, establish connection, send/receive test payload	None
W1	M2	Set up MVVM architecture, Room schema, repositories, payload models	None

W1	M3	UI wireframes, navigation, reusable components, mock chat/map screens	None
W2	M1	Key generation, QR identity handshake, authenticated encrypted transport	W1 P2P working
W2	M2	Local registration/login, profile/contact CRUD, chat persistence, messaging use cases	W1 database and interfaces
W2	M3	Login, friends, QR scanner, chat screens, connection states	W1 UI foundation
W3	M1	GPS/orientation service, location payload transmission	W2 secure messaging
W3	M2	Location data handling, last-known coordinates, distance/bearing logic	W2 messages/repositories
W3	M3	Offline map integration, friend markers, compass, ambient edge glow	W1 UI + location interfaces
W4	All	Integration, testing, UI polishing, permission/error handling, demo preparation	W1–W3

Weekly Acceptance Criteria

- W1: Two physical Android devices exchange text payloads without internet access.
- W2: Two users add each other through QR verification and exchange encrypted chat messages stored in Room.
- W3: A user shares coordinates, and the receiver displays the friend on an offline map with compass direction and distance.
- W4: Complete demo succeeds with no internet access, including reconnecting, message history, and Friend Finder.

Dependency Management & Risk Mitigation

Risk / Blocker	Mitigation
Nearby connection unstable	Validate on two real devices in W1; highest priority
Encryption delays development	Use Nearby's built-in encrypted transport initially; integrate authenticated app-level encryption in W2
Member 3 blocked by backend	Develop UI using mock repositories and sample data
Offline map setup fails	Prepackage a small offline map region; fallback to a coordinate-based radar view
Compass heading inaccurate	Test sensor accuracy and provide calibration/unreliable-heading feedback
Integration takes longer	Freeze new features after W3; dedicate W4 to stability

Suggested Calendar

Given the proposal on November 3 and demos on December 8 and 15, this schedule leaves a useful buffer.

Period	Milestone
Nov 4–10	W1: Foundation and P2P proof of concept
Nov 11–17	W2: Identity and messaging
Nov 18–24	W3: Location and Friend Finder
Nov 25–Dec 1	W4: Integration and finalization

Dec 2–7	Buffer: regression testing and demo rehearsal
Dec 8 / 15	Final demonstrations